

PART 2: COMBINATIONAL LOGIC

LOGIC GATE IMPLEMENTATIONS

SAAD BAIG – DIGITAL LOGIC DESIGN



The animations and contents in these slides are originally adapted from the DLD course slides designed by Dr. Hasan Baig and Mr. Junaid Ahmed Memon. The original slides can be accessed from <https://www.hasanbaig.com/resources>

© Hasan Baig and Junaid Ahmed Memon. All rights reserved. These files and accompanying materials are made available under the terms of “Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) License” and is available at <https://creativecommons.org/licenses/by-nc-nd/4.0/>

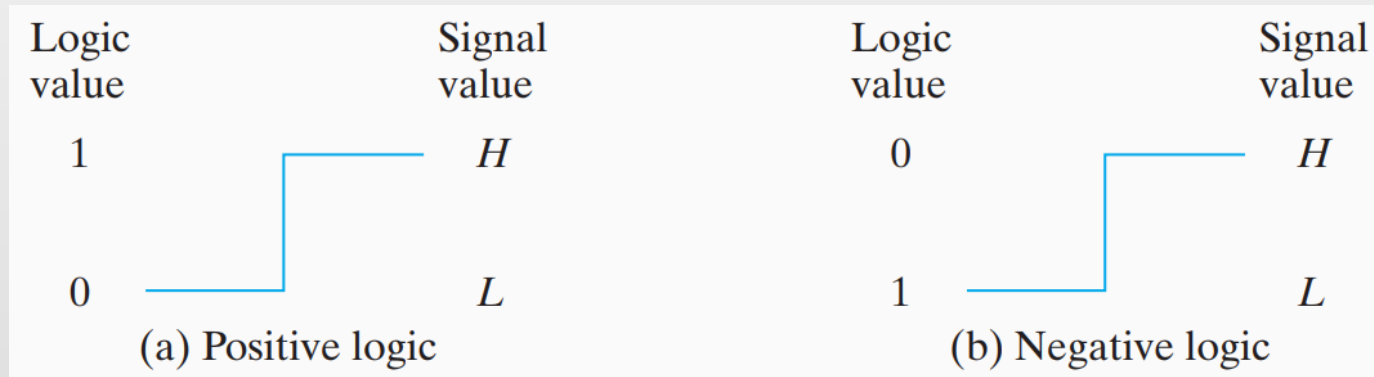
DIGITAL LOGIC GATES



- Since Boolean functions are expressed in terms of AND, OR, and NOT operations, it is easier to implement a Boolean function with these type of gates.
- Still, the possibility of constructing gates for the other logic operations is of practical interest, because of:
 1. The feasibility and economy of producing the gate with physical components.
 2. The possibility of extending the gate to more than two inputs.
 3. The basic properties of the binary operator, such as commutativity and associativity.
 4. The ability of the gate to implement Boolean functions alone or in conjunction with other gates.

POSITIVE AND NEGATIVE LOGIC

- Choosing the high-level H to represent logic 1 defines a **positive logic system**. Choosing the low-level L to represent logic 1 defines a **negative logic system**.



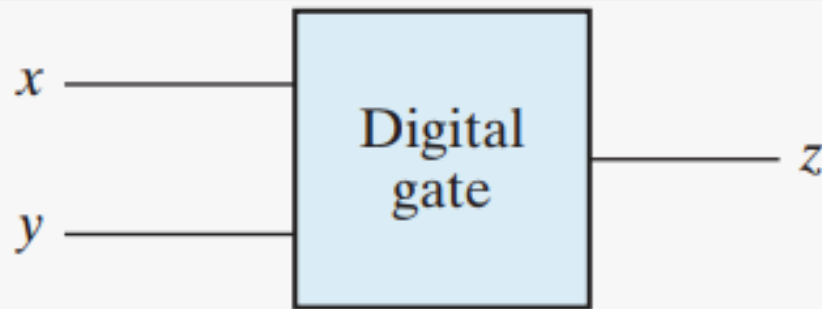
- Hardware digital gates are defined in terms of signal values such as H and L. It is up to the user to decide on a positive or negative logic polarity.

POSITIVE AND NEGATIVE LOGIC

Same physical gates can operate either as a positive-logic AND gate or as a negative-logic OR gate.

x	y	z
L	L	L
L	H	L
H	L	L
H	H	H

(a) Truth table with H and L



(b) Gate block diagram

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

(c) Truth table for positive logic



(d) Positive logic AND gate

x	y	z
1	1	1
1	0	1
0	1	1
0	0	0

(e) Truth table for negative logic



(f) Negative logic OR gate



DIGITAL LOGIC GATES

There are 2^{2^n} functions for n binary variables.

Truth Tables for the 16 Functions of Two Binary Variables

<i>x</i>	<i>y</i>	<i>F</i>₀	<i>F</i>₁	<i>F</i>₂	<i>F</i>₃	<i>F</i>₄	<i>F</i>₅	<i>F</i>₆	<i>F</i>₇	<i>F</i>₈	<i>F</i>₉	<i>F</i>₁₀	<i>F</i>₁₁	<i>F</i>₁₂	<i>F</i>₁₃	<i>F</i>₁₄	<i>F</i>₁₅
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1



Table 2.8
Boolean Expressions for the 16 Functions of Two Variables

Boolean Functions	Operator Symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x , but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y , but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y , but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$(x \oplus y)'$	Equivalence	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y , then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x , then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

1. Constants [2]

2. Unary Operations [4]

3. Binary Operators [10]

DIGITAL LOGIC GATES

EE 172 / CS 120: DIGITAL LOGIC AND DESIGN

Some of these functions are standard gates in digital design:
AND, OR, NAND, NOR and X-OR, X-NOR.

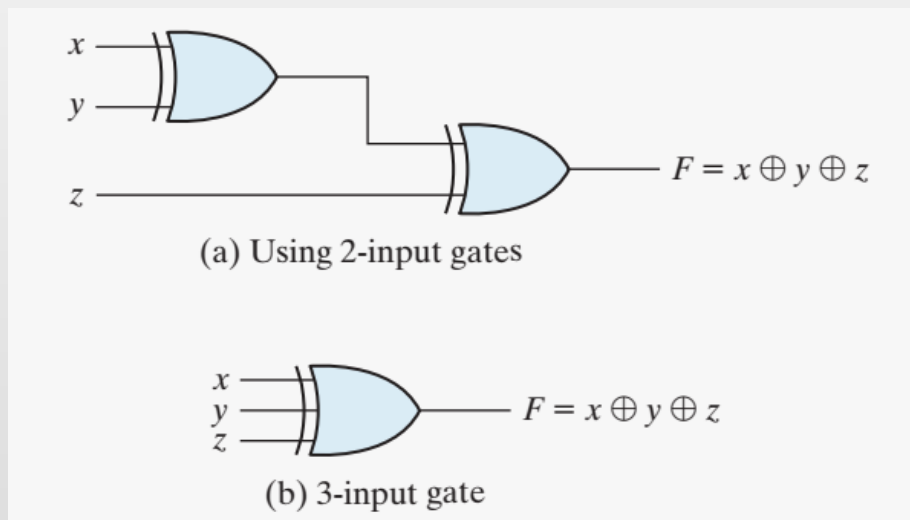


EXTENSION TO MULTIPLE INPUTS

- A gate can be extended to multiple inputs if the binary operation it represents is *commutative* and *associative*.
- **Example:** OR possesses both properties i.e.
 - Commutative: $x + y = y + x$
 - Associative: $(x + y) + z = x + (y + z) = x + y + z$
- OR / AND can be extended to three or more variables.

EXTENSION TO MULTIPLE INPUTS

- X-OR is both commutative and associative and can be extended to more than two inputs.



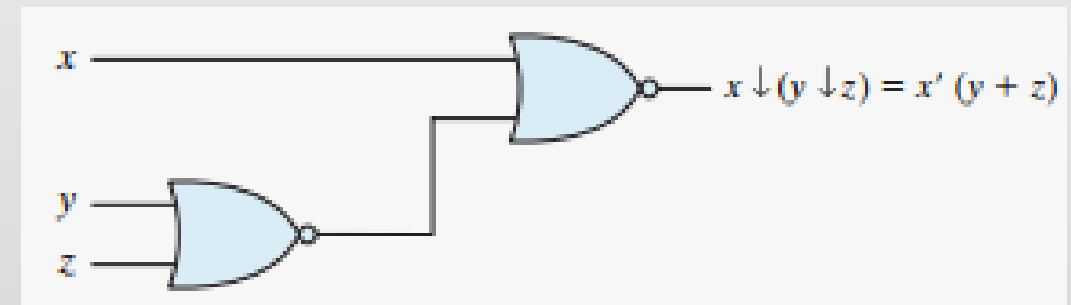
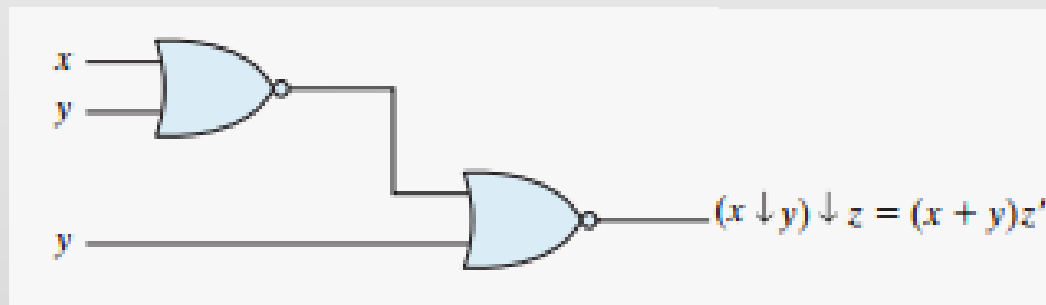
x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(c) Truth table

- X-OR is odd function (i.e., it is equal to 1 if the input variables have an odd number of 1's)

EXTENSION TO MULTIPLE INPUTS

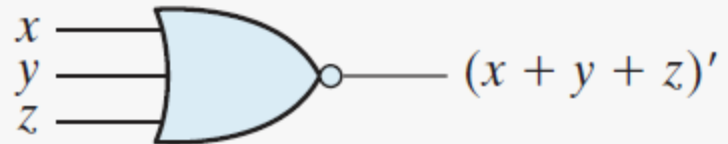
- The NAND and NOR functions are commutative, and their gates can be extended to have more than two inputs, provided that the definition of the operation is modified slightly
- The difficulty is that the NAND and NOR operators are not associative:



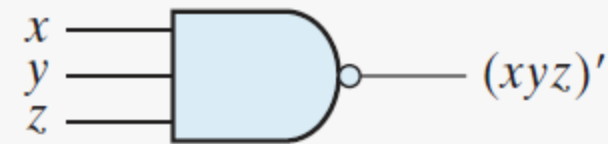
EXTENSION TO MULTIPLE INPUTS

- We define the multiple NOR (or NAND) gate as a complemented OR (or AND) gate:

$$\begin{aligned}x \downarrow y \downarrow z &= (x + y + z)' \\x \uparrow y \uparrow z &= (xyz)'\end{aligned}$$



(a) 3-input NOR gate



(b) 3-input NAND gate

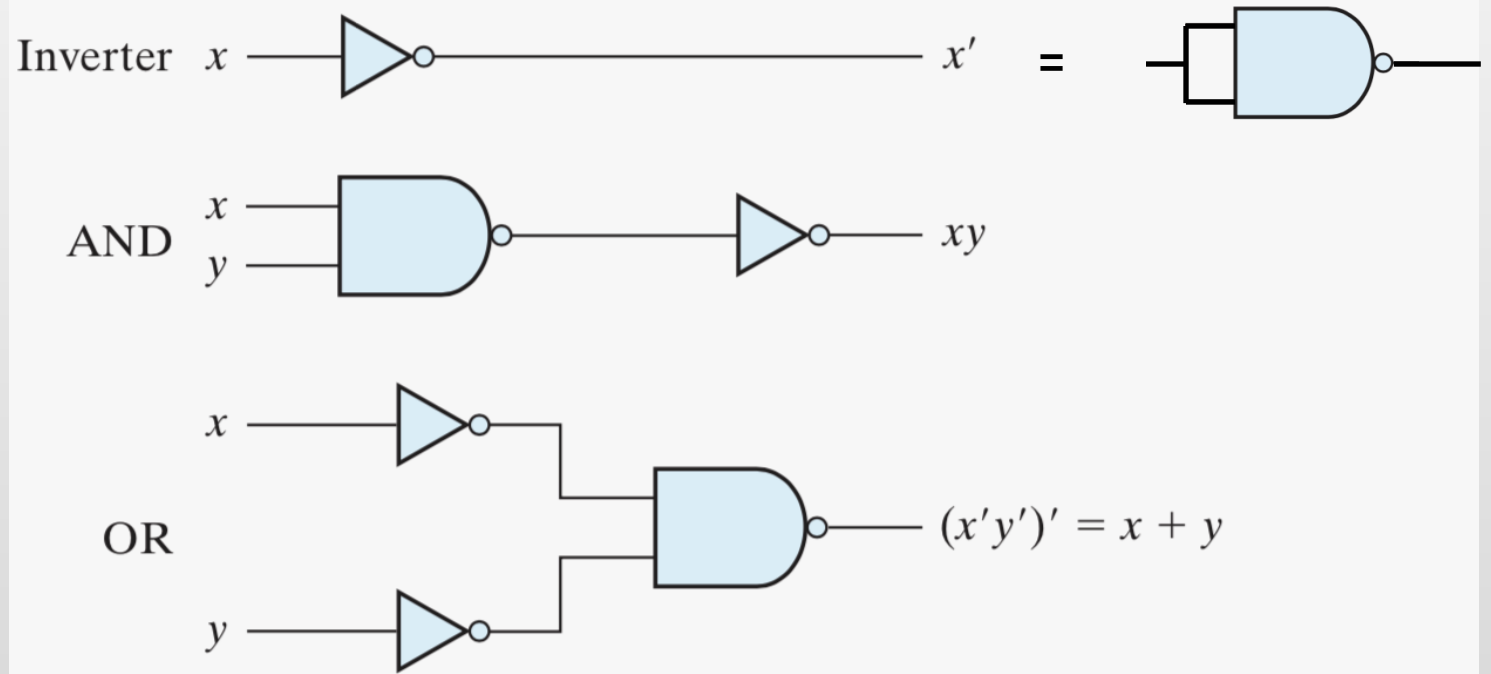


NAND NOR IMPLEMENTATION

- The NAND gate is significant because any Boolean function can be implemented by using a combination of NAND gates. This property is called **functional completeness**.
- NOR gates can also implement any Boolean function.
- Other advantages are:
 - Generally used for circuit design.
 - Easier to fabricate.
 - Basic gates.
 - Other functions can be generated from them.

NAND IMPLEMENTATION

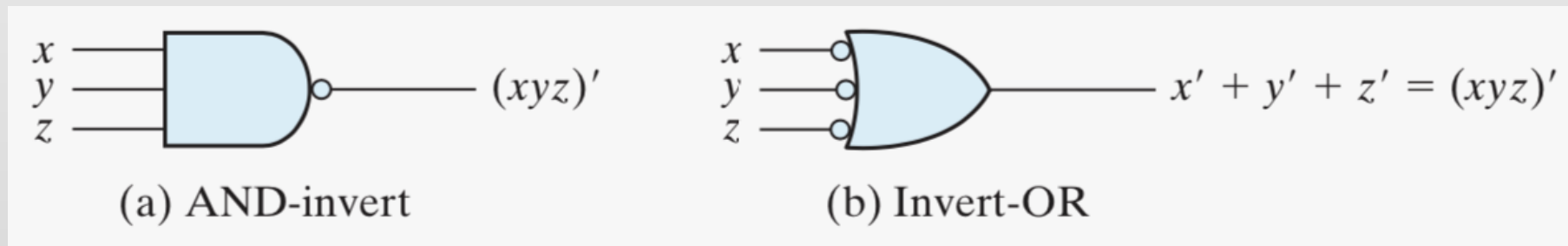
- NAND gate is universal.
- Any digital system can be implemented with it.
- AND, OR and NOT can be implemented with NANDs.



NAND NOR IMPLEMENTATION

A convenient way to implement a Boolean function with NAND gates is to obtain the simplified Boolean function in terms of Boolean operators and then convert the function to NAND logic.

- To facilitate the conversion to NAND logic, it is convenient to define an alternative graphic symbol for the gate.





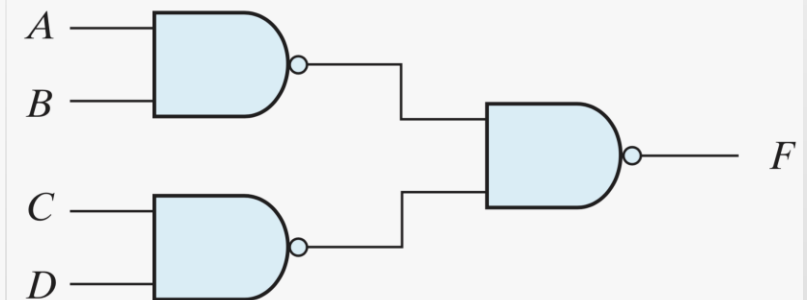
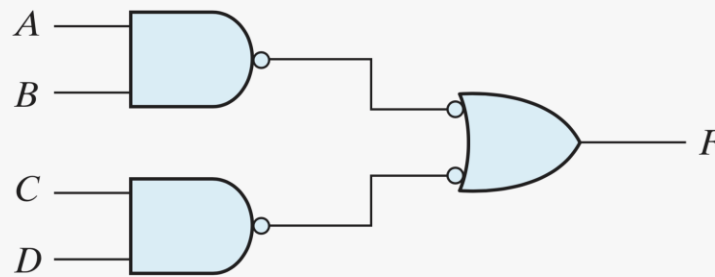
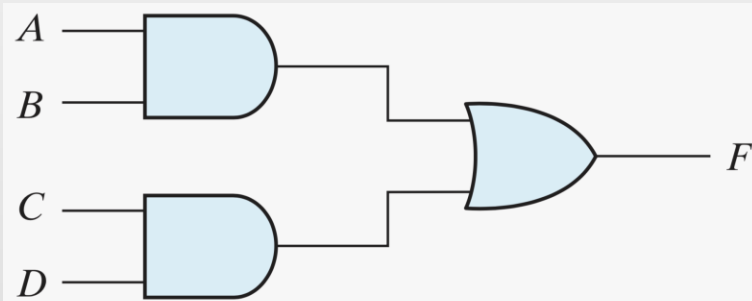
TWO-LEVEL NAND GATE IMPLEMENTATION

The implementation of Boolean functions with NAND gates requires that the functions be in sum-of-products form.

- Put bubbles (inverters) to have two different representations of NAND gate
- Either AND-invert or Invert-OR

TWO-LEVEL NAND GATE IMPLEMENTATION

■ Example: $F = AB + CD$



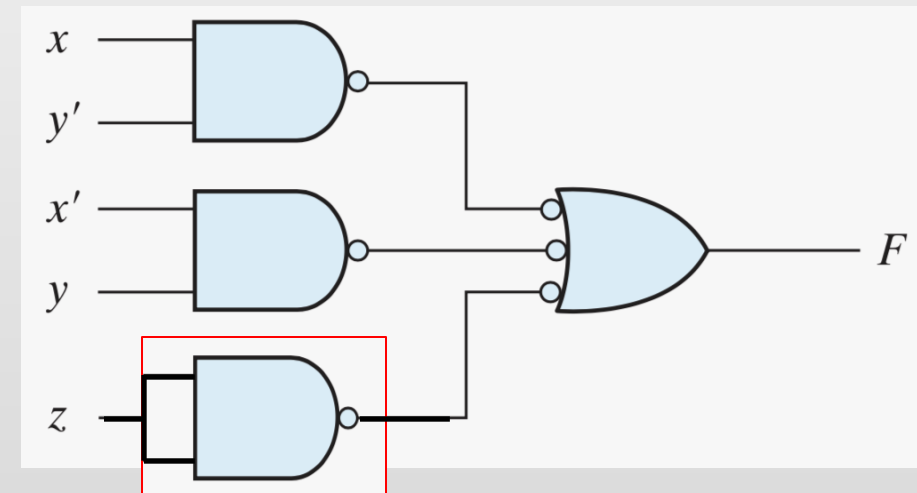
$$F = ((AB)'(CD)')' = AB + CD$$

TWO-LEVEL NAND GATE IMPLEMENTATION

- Implement the Boolean function $F(x, y, z) = \Sigma(1, 2, 3, 4, 5, 7)$ with NAND gates.

x \ yz	yz			
	00	01	11	10
0		1	1	1
1	1	1	1	

$$F = xy' + x'y + z$$



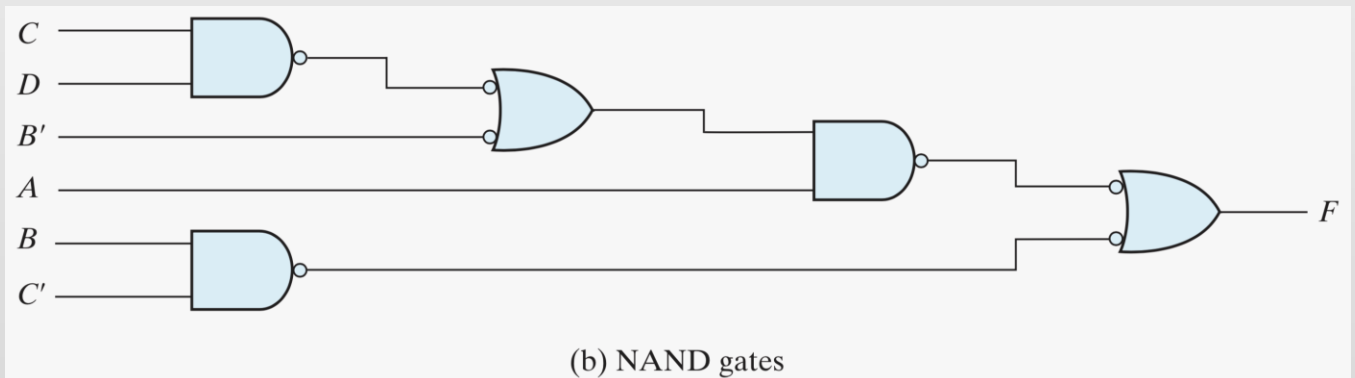
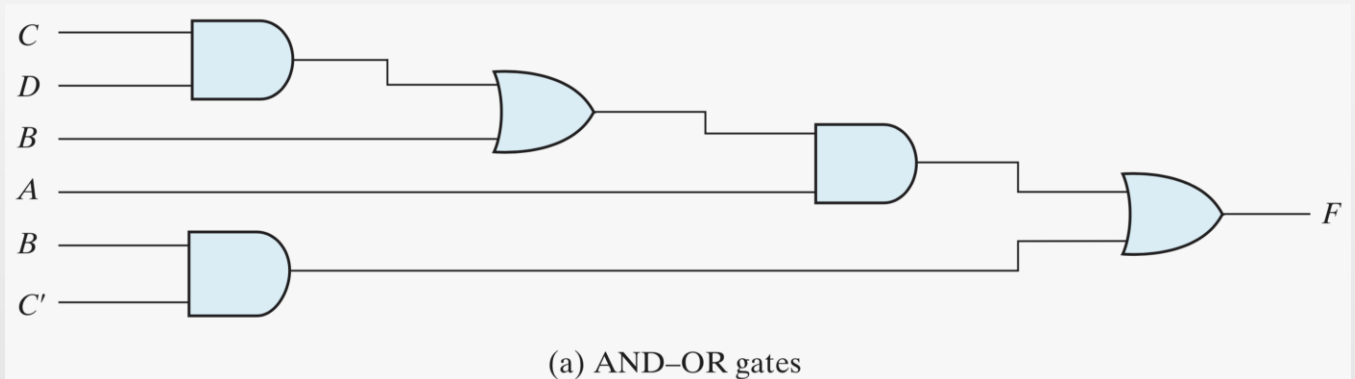


MULTI-LEVEL NAND GATE IMPLEMENTATION

- The standard form of expressing Boolean functions results in a two-level implementation.
- There are occasions when the design of digital systems results in gating structures with three or more levels
- The most common procedure in the design of multilevel circuits is to
 1. Express the function in terms of AND, OR, and complement operations.
 2. Implemented with AND and OR gates.
 3. Converted into an all-NAND circuit.

MULTI-LEVEL NAND GATE IMPLEMENTATION

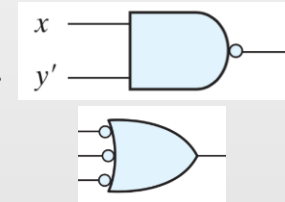
- **Example:** $F = A(CD + B) + BC'$



MULTI-LEVEL NAND GATE IMPLEMENTATION

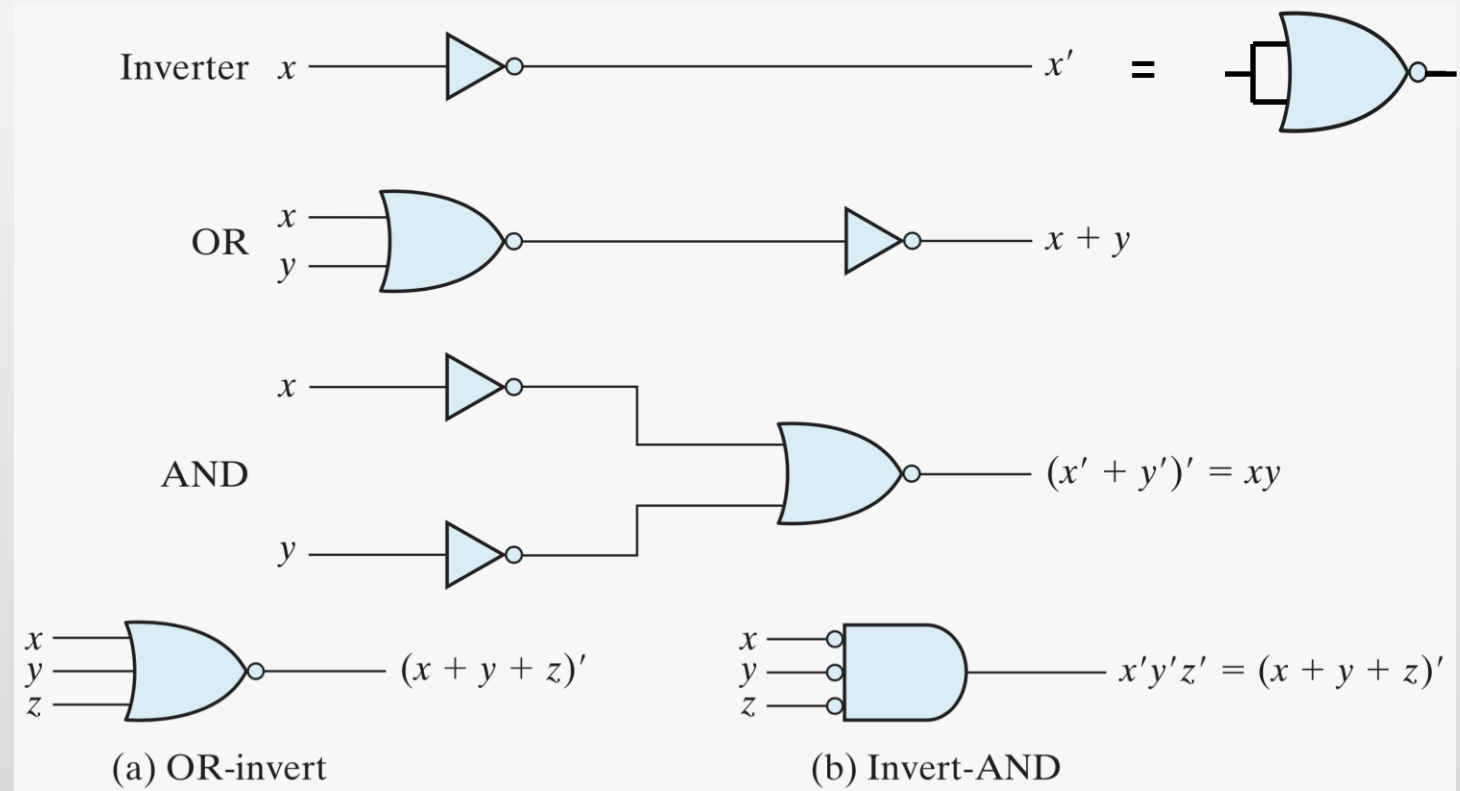
■ Conversion Steps

1. Convert all AND gates to NAND gates with AND-invert graphic symbols.
2. Convert all OR gates to NAND gates with invert-OR graphic symbols.
3. Check all the bubbles in the diagram. For every bubble that is not compensated by another small circle along the same line, insert an inverter (a one-input NAND gate) or complement the input literal.



NOR GATE IMPLEMENTATION

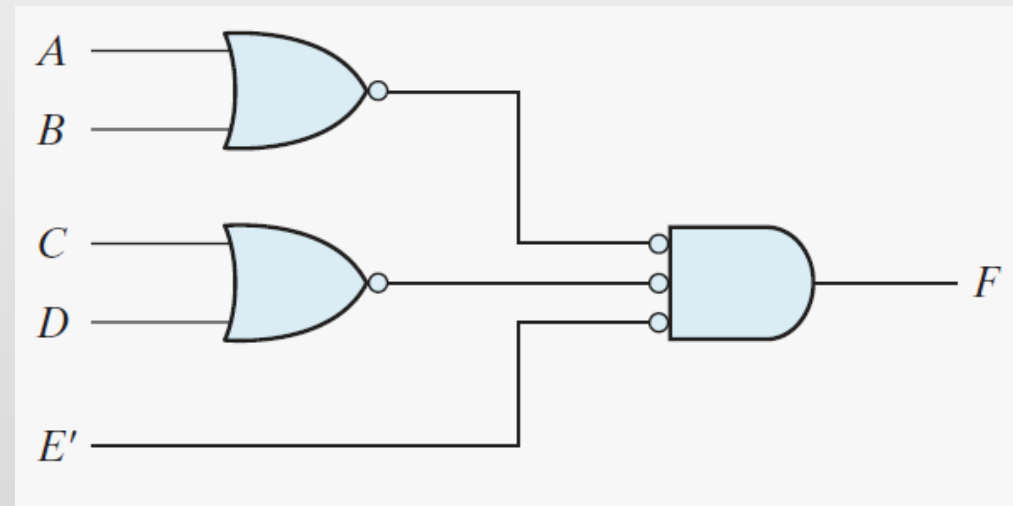
- The NOR gate is another universal gate that can be used to implement any Boolean function



NOR GATE IMPLEMENTATION

A two-level implementation with NOR gates requires that the function be simplified into product-of-sums form.

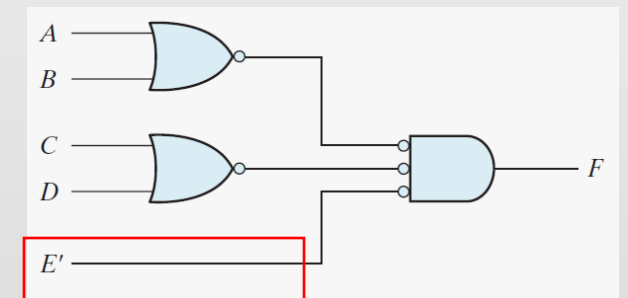
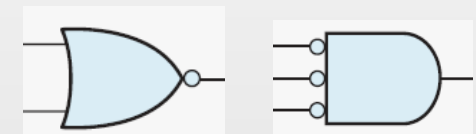
- **Example:** $F = (A + B)(C + D)E$



NOR GATE IMPLEMENTATION

■ Conversion Steps

1. Simplify the function and express it in product-of-sum form.
2. Convert each OR gate to an OR-invert symbol and each AND gate to an invert-AND symbol.
3. Any bubble that is not compensated by another bubble along the same line needs an inverter, or the complementation of the input literal.



$$F = (A + B)(C + D)E$$



NON-DEGENERATE FORMS

- Many two-level combinations of gates are possible. If we assign one type of gate for the first level and one type for the second level, we find that there are 16 possible combinations of two-level forms.
- The same type of gate can be in the first and second levels, as in a NAND–NAND implementation.
- Eight of these combinations are said to be **degenerate forms** because they degenerate to a single operation.
 - **Example:** With AND gates in the first level and an AND gate in the second level, the output of the circuit is merely the AND function of all input variables.



NON-DEGENERATE FORMS

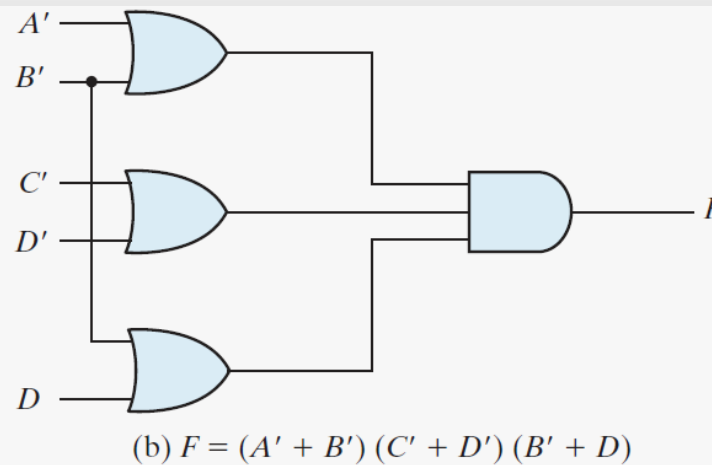
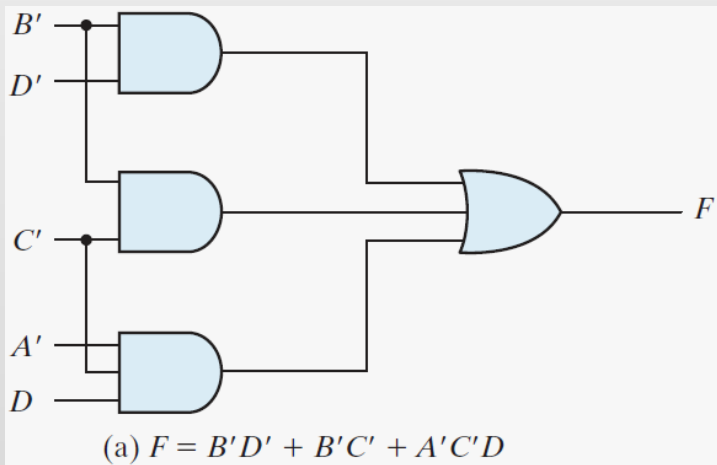
- The remaining eight **nondegenerate forms** produce an implementation in sum-of-products form or product-of-sums form. The eight nondegenerate forms are as follows:

AND–OR	OR–AND
NAND–NAND	NOR–NOR
NOR–OR	NAND–AND
OR–NAND	AND–NOR

← Duals →

AND–OR & OR–AND IMPLEMENTATION

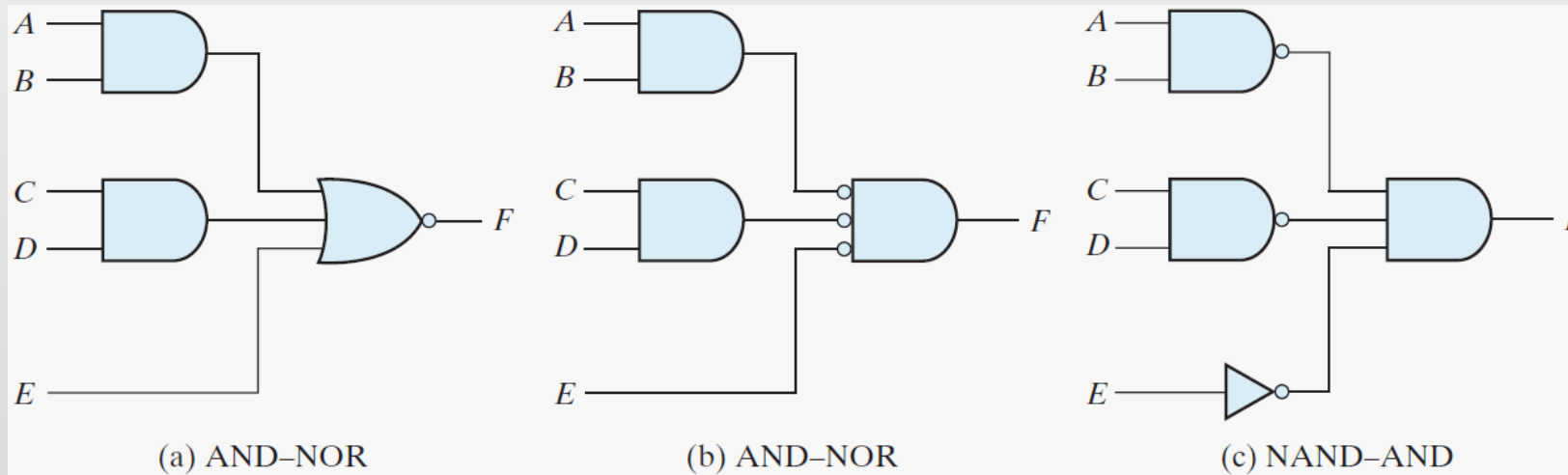
- **Example:** $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$
- SOP: $F = B'D' + B'C + A'C'D$, POS: $F = (A' + B')(C' + D')(B' + D)$
- Two different implementations of the same function.



		yz			
		00	01	11	10
wx	00	m ₀	m ₁	m ₃ 0	m ₂
	01	m ₄ 0	m ₅	m ₇ 0	m ₆ 0
	11	m ₁₂ 0	m ₁₃ 0	m ₁₅ 0	m ₁₄ 0
	10	m ₈	m ₉	m ₁₁ 0	m ₁₀

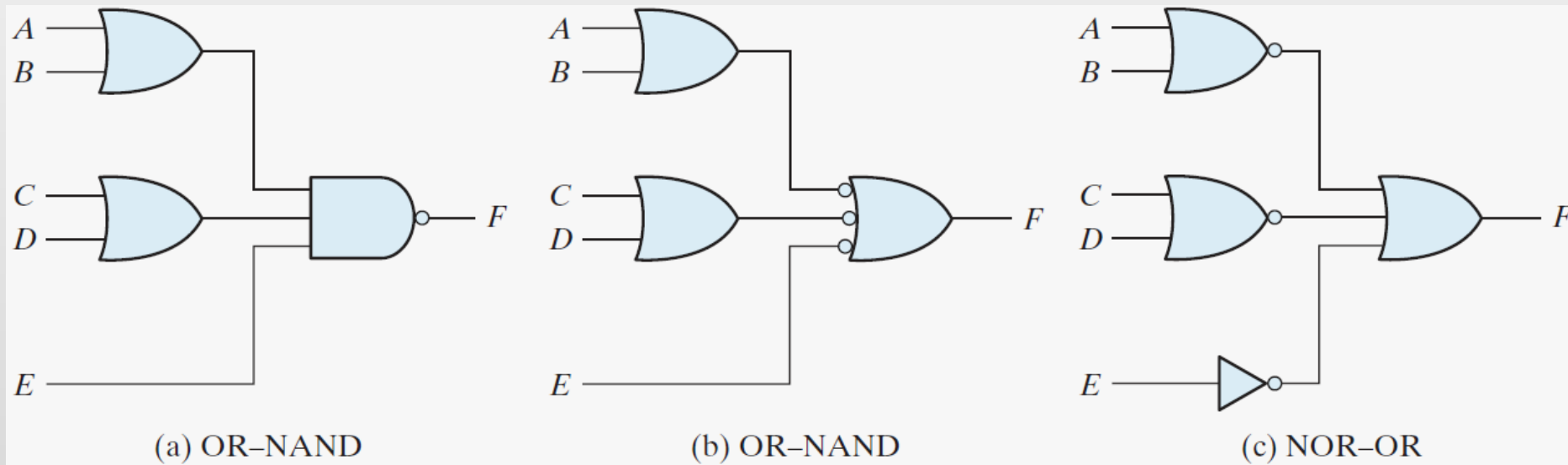
AND-OR-INVERT IMPLEMENTATION

- Both NAND-AND and AND-NOR perform the AND-OR-INVERT function (take AND-OR and complement it).
- **Example:** $F = (AB + CD + E)'$



OR-AND-INVERT IMPLEMENTATION

- The OR-NAND and NOR-OR forms perform the OR-AND-INVERT function.
 - **Example:** $F = [(A + B).(C + D).E]'$





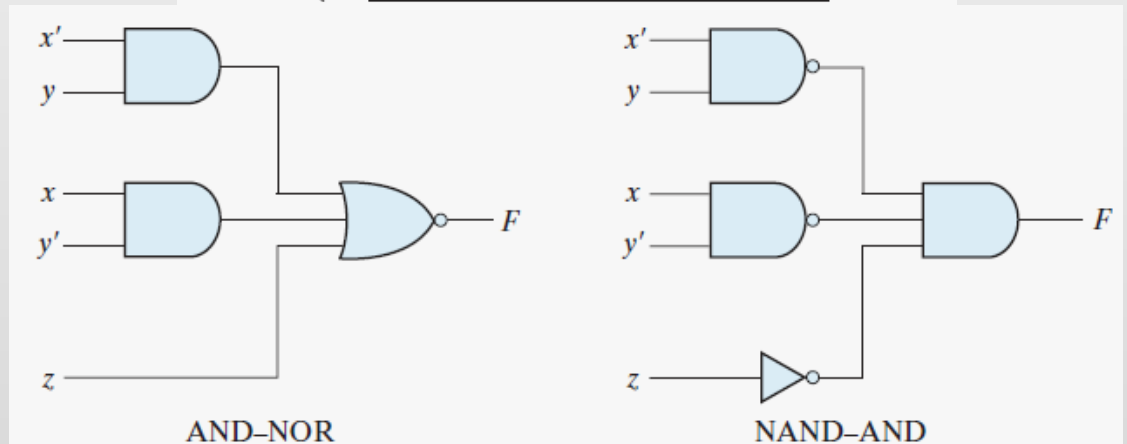
NON-DEGENERATE FORMS

Equivalent Nondegenerate Form		Implements the Function	Simplify F' into	To Get an Output of
(a)	(b)*			
AND-NOR	NAND-AND	AND-OR-INVERT	Sum-of-products form by combining 0's in the map.	F
OR-NAND	NOR-OR	OR-AND-INVERT	Product-of-sums form by combining 1's in the map and then complementing.	F

NON-DEGENERATE FORMS

- **Example:** Implement the function with the AND–OR–INVERT form.
- The complement of the function is simplified into SOP form by combining the 0's in the map: $F' = x'y + xy' + z$
- The normal output for this function can be expressed as $F = (x'y + xy' + z)'$ which is in the AND–OR–INVERT form.

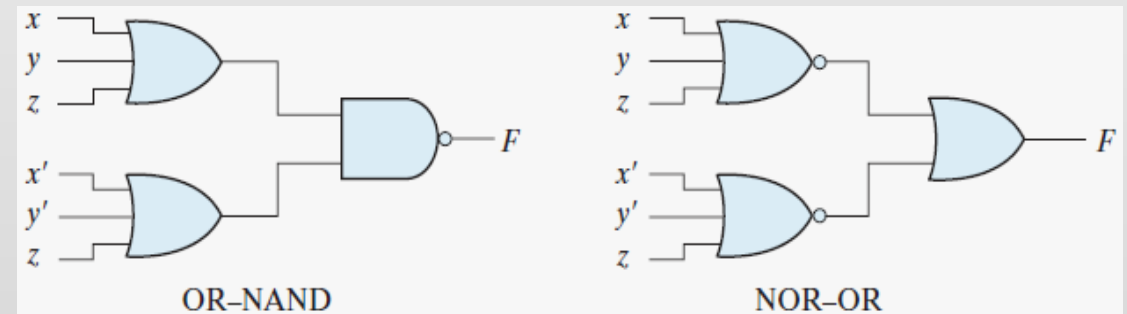
		y			
		00	01	11	10
x \ yz	0	m_0 1	m_1 0	m_3 0	m_2 0
	1	m_4 0	m_5 0	m_7 0	m_6 1



NON-DEGENERATE FORMS

- **Example:** Implement the function with the OR–AND–INVERT form.
- The function is simplified into POS form by combining the 1's in the map: $F' = x'y'z' + xyz'$
- Then we take the complement of the function:
 $F' = (x + y + z)(x' + y' + z)$
- The normal output for this function can be expressed as $F = [(x + y + z)(x' + y' + z)]'$ which is in the OR–AND–INVERT form.

		y			
		00	01	11	10
x \ yz	0	m_0 1	m_1 0	m_3 0	m_2 0
	1	m_4 0	m_5 0	m_7 0	m_6 1





EXCLUSIVE-OR FUNCTION

- X-OR function: $x \oplus y = xy' + x'y$
 - Output is 1 only if x is 1 or y is 1.
 - Output is high when both inputs are different.
- X-NOR function: $(x \oplus y)' = xy + x'y'$
 - Output is 1 only if x and y are both 1 or 0.
 - Output is high when both inputs are same.

- Identities:

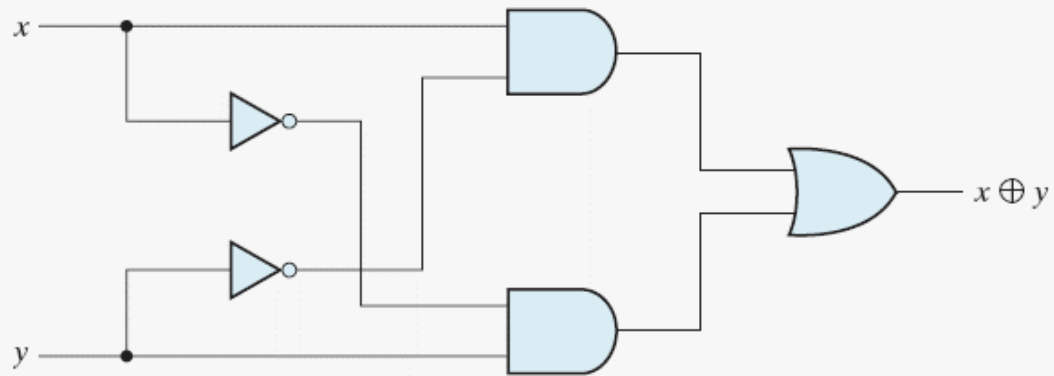
$x \oplus 0 = x$	$x \oplus x = 0$
$x \oplus 1 = x'$	$x \oplus x' = 1$
$x \oplus y' = x' \oplus y = (x \oplus y)'$	

- X-OR operation is commutative and associative:

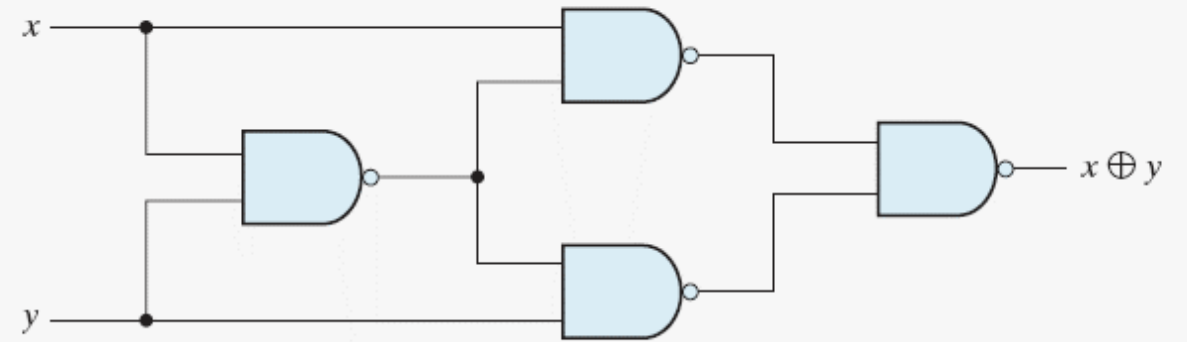
$$A \oplus B = B \oplus A$$

$$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$$

EXCLUSIVE-OR FUNCTION



(a) With AND-OR-NOT gates

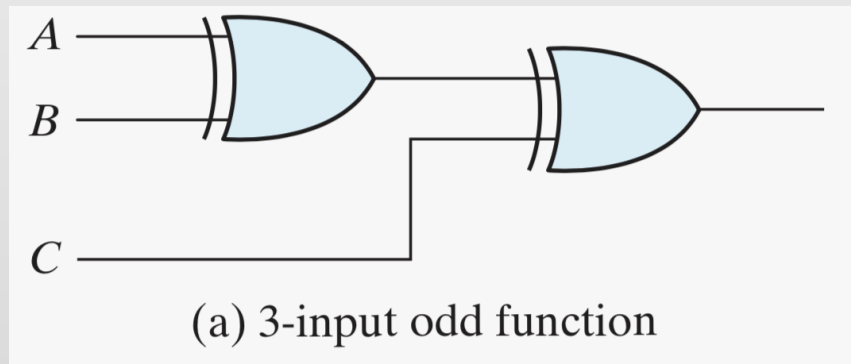


(b) With NAND gates

ODD AND EVEN FUNCTION

- $A \oplus B \oplus C = (AB' + A'B).C' + (AB' + A'B).C = AB'C' + A'BC' + ABC + A'B'C$
- $A \oplus B \oplus C = \sum(1, 2, 4, 7)$

N variable X-OR function is odd function.



		BC			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
			1		1
		1		1	

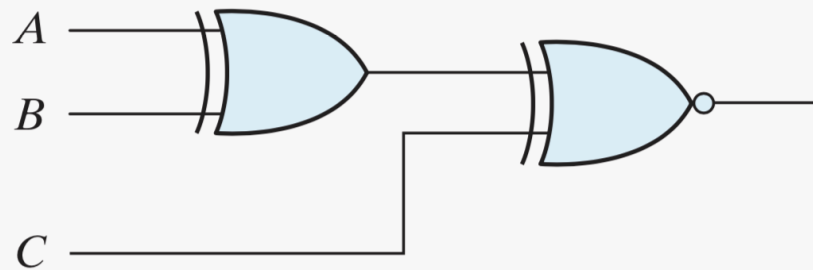
(a) Odd function $F = A \oplus B \oplus C$

		CD			
		00	01	11	10
AB	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}
			1		1
		1		1	

(a) Odd function $F = A \oplus B \oplus C \oplus D$

ODD AND EVEN FUNCTION

N variable X-NOR function is even function.



(b) 3-input even function

		BC			
		00	01	11	10
A	0	m_0 1	m_1	m_3 1	m_2
	1	m_4	m_5 1	m_7	m_6 1

(b) Even function $F = (A \oplus B \oplus C)'$

		CD			
		00	01	11	10
AB	00	m_0 1	m_1	m_3 1	m_2
	01	m_4	m_5 1	m_7	m_6 1
	11	m_{12} 1	m_{13}	m_{15} 1	m_{14}
	10	m_8	m_9 1	m_{11}	m_{10} 1

(b) Even function $F = (A \oplus B \oplus C \oplus D)'$

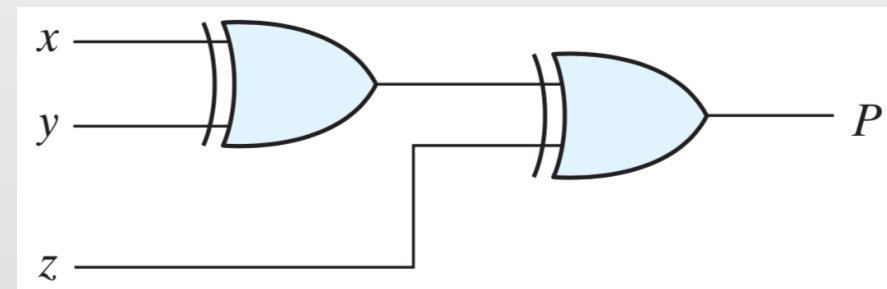


ERROR DETECTION AND CORRECTION

- Exclusive-OR functions are very useful in systems requiring error detection and correction codes. **Parity** is used for the purpose of detecting errors during the transmission of binary information. A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even.
 - The message, including the parity bit, is transmitted and then checked at the receiving end for errors.
 - An error is detected if the checked parity does not correspond with the one transmitted.
 - **Parity Generator:** The circuit that generates the parity bit in the transmitter.
 - **Parity Checker:** The circuit that checks the parity in the receiver.

3-BIT EVEN PARITY GENERATOR

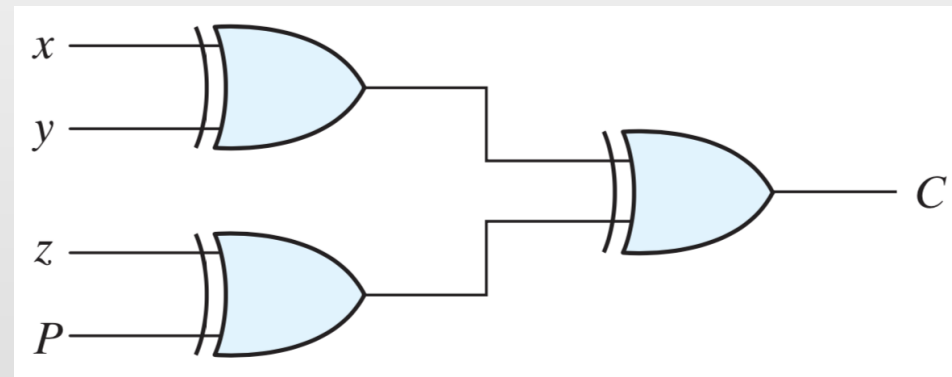
Three-Bit Message			Parity Bit
x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



- 3-bits message + 1-bit parity altogether should have even number of 1's.

3-BIT EVEN PARITY CHECKER

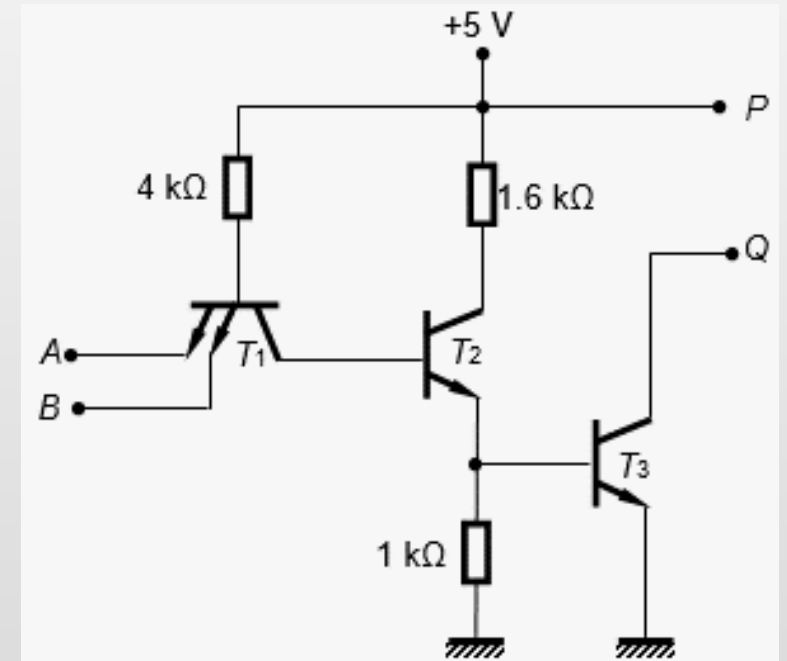
Four Bits Received				Parity Error Check
x	y	z	P	C
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



- Indicate error if the entire 4-bit message does not contain even number of 1's.

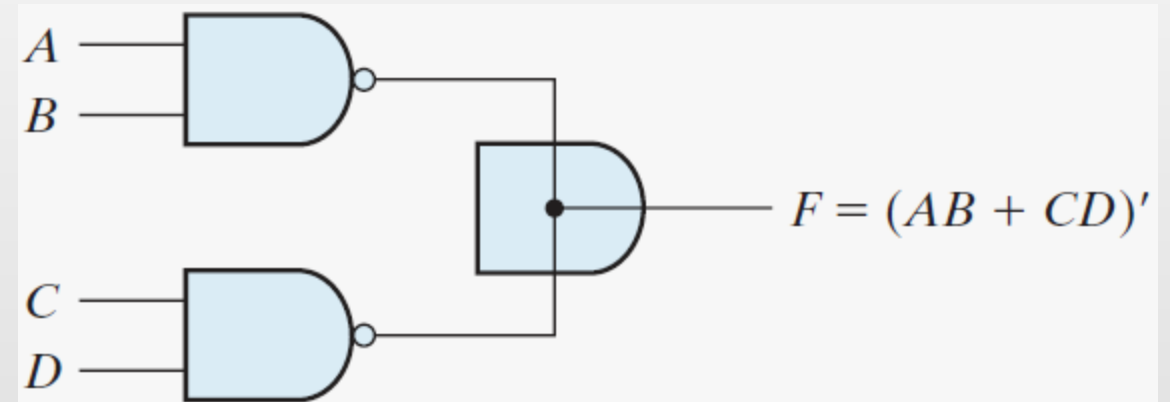
WIRED LOGIC

- The types of gates most often found in integrated circuits are NAND and NOR gates. For this reason, NAND and NOR logic implementations are the most important from a practical point of view.
- Some (but not all) NAND or NOR gates allow the possibility of a wire connection between the outputs of two gates to provide a specific logic function. This type of logic is called *wired logic*.
 - **Example:** Open-collector TTL NAND gates, when tied together, perform wired-AND logic.



WIRED-AND LOGIC

- The AND gate is drawn with the lines going through the center of the gate to distinguish it from a conventional gate.
- The wired-AND gate is not a physical gate, but only a symbol to designate the function obtained from the indicated wired connection.



(a) Wired-AND in open-collector
TTL NAND gates.

(AND-OR-INVERT)

WIRED-OR LOGIC

- Similarly, the NOR outputs of ECL gates can be tied together to perform a wired-OR function.
- A wired-logic gate does not produce a physical second-level gate, since it is just a wire connection.
- The wired connection in the graphic symbol will be omitted in future discussions.

